

PRUEBA PRÁCTICA UNAB

JAVA DESDE CERO HASTA TOMCAT Y JDBC

Java · POO · SQL · PostgreSQL · Oracle · JDBC · JSP/Servlet · Tomcat

Propósito: desarrollar una base suficiente para resolver una prueba presencial sin depender de IA. La prioridad es escribir código pequeño que compile, proteger datos, explicar decisiones y diagnosticar errores.

Material incluido

- Fundamentos explicados paso a paso.
- Laboratorios con archivos iniciales y soluciones separadas.
- Proyecto JDBC real contra PostgreSQL.
- Aplicación WAR con JSP/Servlet para Tomcat.
- Tres simulacros progresivos y rúbricas.

IVÁN DAVID CARDONA MENDOZA

Ingeniero de Sistemas · Especialista en Seguridad Informática

1. Qué puede evaluar la prueba

Por la vacante y el stack mencionado por la directora, la prueba probablemente combinará lectura de requerimientos, Java, SQL, mantenimiento de una aplicación existente y explicación de controles. No asumas que pedirán construir un sistema completo.

Bloque	Qué podrían pedir	Qué demostrar
Java	Clase, método, colección, validación o corrección de error.	Sintaxis, POO, claridad y casos límite.
SQL	Consulta, join, agrupación, actualización o modelo.	Integridad, filtros correctos y seguridad.
JDBC	CRUD o transacción.	PreparedStatement, recursos, commit y rollback.
Web	Servlet/JSP, endpoint o mantenimiento.	HTTP, capas, validación y PRG.
Tomcat	WAR, ruta que falla o log.	Despliegue y diagnóstico.
Seguridad	Entrada maliciosa, perfiles o auditoría.	Mínimo privilegio, trazabilidad y manejo de errores.

Estrategia: primero identifica entrada, salida, reglas y errores. Después crea la estructura mínima, compila y agrega comportamiento por partes. No escribas cien líneas sin compilar.

2. Cómo funciona Java

El archivo `.java` contiene código fuente. `javac` lo convierte en bytecode `.class`, y la JVM ejecuta ese bytecode. Java verifica tipos antes de ejecutar; por eso muchos errores aparecen al compilar.

```
// Archivo Hola.java
public class Hola {
    public static void main(String[] args) {
        System.out.println("Hola UNAB");
    }
}

javac Hola.java
java Hola
```

Reglas que debes recordar

- El nombre del archivo coincide con la clase pública.
- Cada sentencia termina normalmente con punto y coma.
- Java diferencia mayúsculas y minúsculas.
- `String` es un objeto; se compara con `equals()`.
- `null` significa ausencia de referencia y debe validarse.
- Un mensaje de compilación indica archivo, línea y tipo de problema.

Lectura de error: empieza por el primer error. Los siguientes pueden ser consecuencias. Corrige uno, recompila y vuelve a leer.

3. Variables, condiciones y ciclos

```
int cantidad = 30;
double valor = 12500.50;
boolean activo = true;
String nombre = "Iván";

if (activo && cantidad > 0) {
    System.out.println(nombre + ": " + cantidad);
}

for (int i = 0; i < cantidad; i++) {
    System.out.println(i);
}
```

Tipo	Ejemplo	Uso
int / long	int total = 3;	Enteros; long para ids grandes.
double	double valor = 2.5;	Decimales generales. Dinero real suele usar BigDecimal.
boolean	boolean activo = true;	Condición verdadera o falsa.
char	char letra = 'A';	Un carácter.
String	String texto = "hola";	Cadena de caracteres.

Errores frecuentes

```
// Incorrecto
if (estado == "ABIERTO") { }
```

```
// Correcto y seguro ante null constante
if ("ABIERTO".equals(estado)) { }
```

- Usar `=` para asignar y `==` para comparar primitivos.
- Olvidar que un índice de lista comienza en cero.
- Crear un ciclo que nunca modifica su condición.

4. Métodos y validación

Un método recibe parámetros, ejecuta una responsabilidad y puede devolver un resultado. Los nombres deben expresar una acción. Valida condiciones inválidas al inicio para evitar lógica anidada.

```
public static double calcularPago(double horas, double valorHora) {
    if (horas < 0 || valorHora < 0) {
        throw new IllegalArgumentException("Valores negativos");
    }

    double normales = Math.min(horas, 8);
    double extras = Math.max(horas - 8, 0);
    return normales * valorHora
        + extras * valorHora * 1.25;
}
```

Preguntas antes de programar

- ¿Qué recibe el método?
- ¿Qué devuelve?
- ¿Qué valores son inválidos?
- ¿Qué ocurre en cero, mínimo y máximo?
- ¿Puede retornar ausencia? Considera `Optional`.
- ¿La responsabilidad cabe en una frase?

En una prueba, primero escribe la firma y tres ejemplos de entrada/salida. Eso reduce cambios improvisados.

5. Programación orientada a objetos

Una clase agrupa estado y comportamiento. La encapsulación protege invariantes: el objeto no debería poder quedar en un estado inválido por una modificación externa.

```
public final class Incidente {
    private final long id;
    private final String titulo;
    private Estado estado;

    public Incidente(long id, String titulo) {
        if (id <= 0) throw new IllegalArgumentException("Id");
        if (titulo == null || titulo.isBlank()) {
            throw new IllegalArgumentException("Título");
        }
        this.id = id;
        this.titulo = titulo.trim();
        this.estado = Estado.ABIERTO;
    }

    public Estado getEstado() {
        return estado;
    }
}
```

Concepto	Significado práctico
Encapsulación	Atributos privados y cambios controlados por métodos.
Abstracción	Exponer lo necesario y ocultar detalles.
Herencia	Una clase especializa otra; úsala solo si existe relación real "es un".
Polimorfismo	Trabajar contra interfaz y cambiar implementación.
Composición	Un objeto contiene otros; suele ser más flexible que herencia.

6. Interfaces, enums y records

```
public interface IncidenteDao {
    Optional<Incidente> buscarPorId(long id);
    List<Incidente> listar();
}

public enum Estado {
    ABIERTO, EN_PROGRESO, CERRADO
}

public record CrearIncidente(
    String titulo,
    String descripcion,
    Prioridad prioridad) {
}
```

La interfaz define un contrato sin acoplar al consumidor a JDBC o memoria. El enum limita valores válidos. El record es útil para datos inmutables y genera constructor, accesores, `equals`, `hashCode` y `toString`.

Cuándo no usar record

- Cuando el objeto tiene identidad y estado mutable complejo.
- Cuando necesitas ocultar o transformar accesores.
- Cuando representa una entidad JPA con requisitos específicos.

Respuesta de entrevista: "Uso interfaces en límites que pueden cambiar, enums para estados cerrados y records para comandos o respuestas inmutables."

7. Colecciones y Optional

Tipo	Propiedad	Ejemplo de uso
List	Ordenada, admite repetidos.	Incidentes de una consulta.
Set	No admite duplicados.	Permisos o códigos únicos.
Map	Clave asociada a valor.	Conteo por prioridad.
Optional	Resultado que puede no existir.	Buscar por id.

```
List<Incidente> abiertos = incidentes.stream()
    .filter(i -> i.getEstado() == Estado.ABIERTO)
    .toList();

Map<Prioridad, Long> conteo = incidentes.stream()
    .collect(Collectors.groupingBy(
        Incidente::getPrioridad,
        Collectors.counting()));

Incidente incidente = buscarPorId(id)
    .orElseThrow(() -> new NoEncontradoException(id));
```

Reglas

- No expongas una lista interna mutable; devuelve copia.
- Usa 'Set' cuando la unicidad sea parte del problema.
- No uses 'Optional' como sustituto universal de validación.
- Puedes resolver con bucles si aún no dominas Streams; claridad primero.

8. Excepciones y diagnóstico

```
try {
    service.crear(comando);
} catch (IllegalArgumentException e) {
    System.out.println("Dato inválido: " + e.getMessage());
} catch (SQLException e) {
    System.err.println("Error de persistencia");
    throw e;
}
```

Las excepciones de validación representan errores esperables del usuario. `SQLException` representa un problema técnico que debe registrarse con contexto y causa, sin mostrar SQL ni credenciales al usuario.

Cómo leer una traza

- Identifica tipo y mensaje.
- Busca el primer `Caused by` relevante.
- Encuentra la primera línea de tu código.
- Revisa datos de entrada y cambio reciente.
- Corrige la causa, no ocultes el síntoma.

Nunca uses `catch (Exception e) {}`. Silenciar un error hace que la aplicación continúe en un estado desconocido.

9. Modelo relacional y restricciones

```
CREATE TABLE incidente (  
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    titulo VARCHAR(120) NOT NULL,  
    estado VARCHAR(20) NOT NULL,  
    solicitante_id BIGINT NOT NULL,  
    CONSTRAINT fk_solicitante  
        FOREIGN KEY (solicitante_id) REFERENCES usuario_tic(id),  
    CONSTRAINT ck_estado  
        CHECK (estado IN ('ABIERTO', 'EN_PROGRESO', 'CERRADO'))  
);
```

Restricción	Protege
PRIMARY KEY	Identidad única y no nula.
FOREIGN KEY	Relaciones con registros existentes.
NOT NULL	Campos obligatorios.
UNIQUE	Valores sin duplicar.
CHECK	Dominio o regla simple.

Las validaciones de Java mejoran la experiencia, pero la base es la última barrera de integridad. Ambas capas deben proteger las reglas críticas.

10. SELECT, JOIN y agregación

```
SELECT i.id, i.titulo, u.nombre AS solicitante
FROM incidente i
JOIN usuario_tic u ON u.id = i.solicitante_id
WHERE i.estado = 'ABIERTO'
ORDER BY i.fecha_creacion DESC;
```

```
SELECT u.nombre, COUNT(i.id) AS total
FROM usuario_tic u
LEFT JOIN incidente i ON i.tecnico_id = u.id
WHERE u.rol = 'TECNICO'
GROUP BY u.id, u.nombre;
```

Elemento	Uso
WHERE	Filtrar filas antes de agrupar.
GROUP BY	Formar grupos para COUNT, SUM, AVG.
HAVING	Filtrar grupos después de agrupar.
INNER JOIN	Solo coincidencias.
LEFT JOIN	Todas las filas izquierdas, exista o no coincidencia.
NOT EXISTS	Comprobar ausencia de relación.

- Usa alias claros y califica columnas ambiguas.
- Filtra el estado en la condición correcta antes de contar.

11. Transacciones, índices y concurrencia

```
BEGIN;

SELECT * FROM equipo WHERE id = 10 FOR UPDATE;
INSERT INTO prestamo (...);
UPDATE equipo SET disponible = FALSE WHERE id = 10;

COMMIT;
-- Ante error: ROLLBACK;
```

Una transacción agrupa cambios que deben ocurrir todos o ninguno. 'FOR UPDATE' bloquea la fila para evitar que dos operaciones presten el mismo equipo simultáneamente.

Control optimista

```
UPDATE incidente
SET estado = ?, version = version + 1
WHERE id = ? AND version = ?;
```

- Si actualiza una fila: éxito.
- Si actualiza cero: otro usuario cambió el registro o ya no existe.

Índices

- Aceleran filtros, joins y ordenamientos usados con frecuencia.
- Consumen espacio y agregan costo a INSERT/UPDATE/DELETE.
- Se definen a partir de consultas reales y se validan con plan de ejecución.

12. PostgreSQL y Oracle

Tema	PostgreSQL	Oracle
Entero grande	BIGINT	NUMBER(19)
Cadena	VARCHAR	VARCHAR2
Booleano tabla	BOOLEAN	NUMBER(1) o CHAR(1)
Texto largo	TEXT	CLOB
Id	IDENTITY	IDENTITY o SEQUENCE
Procedimientos	PL/pgSQL	PL/SQL
URL JDBC	jdbc:postgresql://...	jdbc:oracle:thin:@//...

Particularidades que suelen preguntar

- Oracle trata cadena vacía como `NULL`; PostgreSQL las diferencia.
- PostgreSQL ofrece `ILIKE`; para portabilidad usa `LOWER(campo) LIKE LOWER(?)`.
- Oracle tradicional usa secuencias; versiones modernas admiten identity.
- Ambos soportan transacciones, joins, restricciones y `FETCH FIRST`.

No afirmes dominar Oracle. Explica que tus fundamentos SQL son transferibles y nombra las diferencias que revisarías antes de modificar producción.

13. JDBC paso a paso

```
String sql = "SELECT id, titulo FROM incidente WHERE id = ?";

try (Connection cn = factory.abrir();
    PreparedStatement ps = cn.prepareStatement(sql)) {

    ps.setLong(1, id);

    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) {
            return new Incidente(
                rs.getLong("id"),
                rs.getString("titulo"));
        }
        return null;
    }
}
```

Responsabilidad de cada objeto

- `Connection`: sesión con la base y límite transaccional.
- `PreparedStatement`: SQL precompilado y parámetros separados del texto.
- `ResultSet`: cursor de resultados; `next()` avanza fila.
- `try-with-resources`: cierra incluso si ocurre excepción.

Por qué evita inyección

El texto del usuario se envía como dato. No puede cerrar comillas y transformar la estructura SQL. Nunca reemplaces parámetros concatenando cadenas.

14. Transacciones JDBC

```
try (Connection cn = factory.abrir()) {
    boolean original = cn.getAutoCommit();
    try {
        cn.setAutoCommit(false);

        // DAO 1 usando cn
        // DAO 2 usando la misma cn

        cn.commit();
    } catch (Exception error) {
        try {
            cn.rollback();
        } catch (SQLException rollbackError) {
            error.addSuppressed(rollbackError);
        }
        throw error;
    } finally {
        cn.setAutoCommit(original);
    }
}
```

Errores críticos

- Cada DAO abre una conexión diferente.
- Se hace commit después de capturar un error.
- Se olvida rollback.
- Se devuelve la conexión al pool con autoCommit alterado.
- Se cierra una conexión cuya propiedad corresponde al servicio.
- No se comprueba el número de filas afectadas.

La transacción pertenece al caso de uso, no a una sentencia aislada. Por eso normalmente la coordina el servicio.

15. Maven y estructura de proyecto

```
proyecto/  
  pom.xml  
  src/main/java/  
  src/main/resources/  
  src/main/webapp/  
  src/test/java/  
  
mvn clean      # elimina target  
mvn compile    # compila main  
mvn test       # compila y ejecuta pruebas  
mvn package    # genera JAR o WAR
```

Partes del pom

- ``groupId``: organización o espacio de nombres.
- ``artifactId``: nombre del proyecto.
- ``version``: versión del artefacto.
- ``packaging``: jar o war.
- ``dependencies``: librerías y alcance.
- Servlet API usa ``provided`` porque Tomcat la entrega.

Si Maven falla, lee si el problema es compilación, prueba, descarga o plugin. No borres código al azar.

16. HTTP, Servlet y JSP

Método	Uso habitual
GET	Consultar; no debe modificar estado.
POST	Crear o ejecutar acción.
PUT/PATCH	Actualizar en API REST.
DELETE	Eliminar en API REST.

```
@WebServlet("/incidencias")
public class IncidenciaServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req,
        HttpServletResponse resp)
        throws ServletException, IOException {
        req.setAttribute("incidencias", service.listar());
        req.getRequestDispatcher(
            "/WEB-INF/views/incidencias.jsp"
        ).forward(req, resp);
    }
}
```

Separación

- Servlet: HTTP, parámetros, status, forward/redirect.
- Service: reglas, permisos y transacciones.
- DAO: SQL y mapeo.
- JSP: presentación escapada; sin SQL ni lógica de negocio.

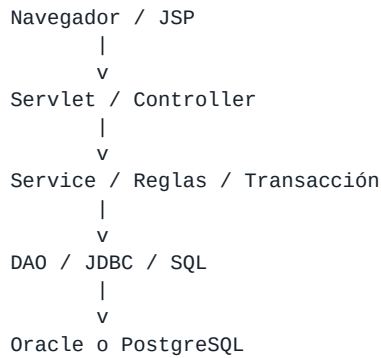
17. Tomcat y WAR

Tomcat es un contenedor web. Carga el WAR, crea un contexto y administra Servlets, filtros, sesiones y JSP.

```
mvn clean test package
cp target/incidentes.war $CATALINA_HOME/webapps/
$CATALINA_HOME/bin/startup.sh
tail -f $CATALINA_HOME/logs/catalina.out
```

Síntoma	Revisión
404	Context path, mapping, despliegue y log de arranque.
500	Primera excepción, Caused by, parámetros y JDBC.
ClassNotFoundException Servlet	Tomcat 9 javax vs Tomcat 10/11 jakarta.
Acentos dañados	UTF-8 antes de leer parámetros, JSP y base.
Conexiones agotadas	Cierre de recursos, pool y transacciones.
Compatibilidad: Tomcat 9 usa `javax.servlet`; Tomcat 10 y 11 usan `jakarta.servlet`. No se mezclan.	

18. Arquitectura de tres capas



La arquitectura separa motivos de cambio. Una regla de transición no pertenece al Servlet ni al DAO. Una consulta SQL no pertenece a la JSP. Esta separación facilita pruebas y mantenimiento.

Cómo explicarla en la prueba

"El controlador traduce HTTP, el servicio aplica reglas y coordina la transacción, y el DAO encapsula persistencia. Así puedo probar reglas sin Tomcat y cambiar detalles de base sin reescribir la interfaz."

- Modelo/DTO: datos que cruzan capas.
- Filtro: preocupaciones transversales como UTF-8, sesión o seguridad.
- Configuración: fuera del código y separada por ambiente.

19. Seguridad mínima

Riesgo	Control
Inyección SQL	PreparedStatement y permisos mínimos.
XSS	Escapar al mostrar con c:out; CSP como defensa adicional.
CSRF	Token de sesión en operaciones POST.
Acceso indebido	Autenticación, roles y autorización por acción.
Secreto expuesto	Variables de entorno o gestor de secretos; no Git.
Cambio sin evidencia	Usuario, fecha, acción, resultado y correlación.
Pérdida de datos	Respaldo verificado y plan de reversión.

Los logs no deben contener

- Contraseñas.
- Tokens completos.
- Cadenas de conexión con credenciales.
- Datos personales innecesarios.

Tu plus como especialista no es agregar controles sin contexto. Es identificar riesgo, elegir un control proporcional y comprobar que no rompe el proceso.

20. Pruebas

Tipo	Qué valida	Ejemplo
Unitaria	Regla aislada.	Transición de estado.
Integración	Componentes reales.	DAO contra base de prueba.
Funcional	Requerimiento.	Crear incidencia desde formulario.
Regresión	Que lo anterior siga funcionando.	Suite antes de liberar.
Aceptación	Necesidad del usuario.	Validación con responsable.

```
@Test
void noPermiteCerrarDirectamente() {
    Incidente incidente = nuevoAbierto();

    assertThrows(
        IllegalStateException.class,
        () -> incidente.cambiarEstado(Estado.CERRADO)
    );
}
```

Casos mínimos

- Camino correcto.
- Campo vacío o nulo.
- Longitud mínima y máxima.
- Estado inválido.
- Registro inexistente.
- Entrada maliciosa.

21. Sprint semanal y trabajo en equipo

Un sprint es un periodo corto para alcanzar un objetivo verificable. Si el equipo revisa avances cada semana, tu trabajo debe poder integrarse y demostrarse; no basta con reportar porcentaje.

Momento	Propósito	Evidencia
Refinamiento	Aclarar alcance, reglas, dependencias y riesgos.	Historia y criterios de aceptación.
Planificación	Elegir objetivo y trabajo alcanzable.	Backlog del sprint y responsables.
Ejecución	Desarrollar en cambios pequeños y coordinados.	Rama, commits y pruebas.
Seguimiento	Exponer avance, bloqueo e impacto.	Funcionalidad ejecutable, no solo porcentaje.
Review	Demostrar valor y recibir retroalimentación.	Criterios cumplidos y pendientes visibles.
Retrospectiva	Mejorar la forma de trabajar.	Acción concreta para el siguiente sprint.

Historia de usuario

Como técnico TIC, quiero cambiar el estado de una incidencia para reflejar el avance.

Criterios: ABIERTA a EN_PROGRESO; EN_PROGRESO a CERRADA; otras transiciones se rechazan; el cambio queda auditado.

- Definition of Done: compila, cumple criterios, tiene pruebas, está revisada, integrada y documentada.

22. Git para integrar trabajo

```
git switch main
git pull
git switch -c feature/us-02-estados

git status
git add .
git commit -m "Agregar transición de estados"

git switch main
git merge --no-ff feature/us-02-estados
mvn clean test
```

Principios

- Una rama resuelve una historia o cambio coherente.
- Los commits son pequeños, compilables y explican intención.
- Antes de integrar, actualiza tu rama y ejecuta pruebas.
- La revisión evalúa lógica, seguridad, pruebas y mantenibilidad; no solo formato.
- Después del merge se ejecuta regresión sobre el sistema integrado.
- Un conflicto textual puede ocultar un conflicto funcional entre contratos.

Resolver un conflicto

```
git status
# abrir archivo y entender ambas versiones
# conservar el comportamiento correcto
mvn test
git add archivo
git commit
```

No resuelvas escogiendo automáticamente "mi versión". Revisa consumidores, contrato compartido, base de datos y pruebas antes de decidir.

23. Sprint no es Spring

Término	Significado
Sprint	Ciclo de trabajo de Scrum con objetivo, backlog, review y retrospectiva.
Spring Framework	Ecosistema Java con inyección de dependencias, MVC, acceso a datos y otros módulos.
Spring Boot	Forma rápida de configurar una aplicación Spring, normalmente con servidor embebido.
Tomcat	Contenedor web que puede ejecutar WAR clásicos o servir como contenedor embebido en Spring Boot.

Arquitectura equivalente en Spring

```
@RestController
class IncidenciaController { ... }

@Service
class IncidenciaService { ... }

@Repository
class IncidenciaRepository { ... }
```

Los fundamentos practicados se transfieren: HTTP, validación, servicio, transacción, SQL, pruebas y seguridad. Spring reduce configuración, pero no reemplaza entender esas responsabilidades.

Pregunta que debes hacer cuando confirmen la prueba: "¿El desarrollo será Java/JSP sobre Tomcat o utilizan Spring/Spring Boot?". La preparación puede ajustarse en cuanto conozcas la respuesta.

24. Práctica final de integración

Trabaja sobre una copia de `04\_web\_tomcat`. Simularás tres integrantes y unirás sus cambios al final.

Rama	Historia	Entrega
feature/us-01-filtros	Filtrar por estado y prioridad.	Servicio, formulario y pruebas.
feature/us-02-estados	Cambiar estado con reglas.	POST, validación y prueba.
feature/us-03-auditoria	Registrar cada cambio.	Modelo, persistencia e historial.

Orden

- Acordar contrato de estados, métodos y HTTP.
- Crear ramas desde la misma base.
- Implementar y probar cada historia por separado.
- Simular review explicando avance, bloqueo y riesgo.
- Integrar una rama a la vez.
- Ejecutar pruebas después de cada merge.
- Resolver un conflicto intencional en IncidenciaService.
- Construir WAR y desplegar en Tomcat.
- Demostrar creación, filtro, transición e historial.

Material: `06\_integracion\_equipo/README.md`, `BACKLOG\_SPRINT.md` y `CONTRATOS\_INTEGRACION.md`.

25. Calendario del 21 al 31 de julio

Fecha	Trabajo principal	Resultado
Mar. 21	Instalación, guía 1–8 y fundamentos.	Ejercicios Java compilados.
Mié. 22	SQL PostgreSQL y diferencias Oracle.	15 consultas y esquema explicable.
Jue. 23	JDBC, PreparedStatement y transacciones.	Proyecto JDBC ejecutado.
Vie. 24	Servlet, JSP, Maven y Tomcat.	WAR desplegado localmente.
Sáb. 25	Simulacro 1 y corrección.	Puntaje, errores y segunda versión.
Dom. 26	Git, sprint y práctica de integración.	Tres ramas integradas.
Lun. 27	Simulacro 2 sin ayuda.	CRUD JDBC cronometrado.
28–30	Si aún no es la prueba: repetir debilidades y simulacro 3.	Aplicación web bajo tiempo.
Día anterior	Repaso corto, comandos y descanso.	Entorno y documentos listos.

Si la prueba es el 27, no intentes terminar el simulacro 3 completo: prioriza fundamentos, SQL, JDBC y explicar la arquitectura. Si es del 28 al 31, usa los días adicionales para integración y Tomcat.

26. Plan de práctica por sesiones

Sesión	Trabajo	Resultado
1	Guía 2–7 + laboratorio fundamentos.	Clases, métodos y colecciones sin copiar.
2	Guía 9–12 + 15 consultas SQL.	DDL, joins, grupos y transacción.
3	Guía 13–14 + proyecto JDBC.	CRUD y cambio de estado transaccional.
4	Guía 15–18 + WAR Tomcat.	Servlet, JSP, capas y despliegue.
5	Guía 19–20 + simulacro 1.	Validación, seguridad y prueba cronometrada.
6	Simulacro 2.	JDBC sin ayuda.
7	Simulacro 3 o versión reducida.	Aplicación web explicable.

En cada sesión

- 20 minutos de lectura.
- 60–120 minutos de código sin solución.
- 15 minutos explicando en voz alta.
- Comparar con solución y registrar tres errores.

No avances porque “entendiste al leer”. Avanza cuando puedas escribir y explicar sin mirar.

27. Hoja de memoria para la prueba

JAVA

```
if (x == null) { ... }
"texto".equals(valor)
List<T> lista = new ArrayList<>();
Map<K,V> mapa = new HashMap<>();
Optional<T> resultado
try-with-resources
```

SQL

```
SELECT ... FROM ... JOIN ... ON ...
WHERE ... GROUP BY ... HAVING ... ORDER BY ...
INSERT INTO ... VALUES ...
UPDATE ... SET ... WHERE ...
BEGIN; COMMIT; ROLLBACK;
```

JDBC

```
Connection
PreparedStatement
setString / setLong
executeQuery / executeUpdate
ResultSet.next()
commit / rollback
```

WEB

```
GET consulta
POST modifica
Servlet -> Service -> DAO
forward si validación falla
redirect después de POST correcto
WAR -> Tomcat webapps
```

Última regla: compila primero una versión pequeña. Después agrega validación, persistencia y seguridad. Deja comentarios claros para lo que no alcances.

28. Checklist de entrega

- [] El proyecto compila.
- [] Ejecuté al menos un caso válido.
- [] Probé un caso inválido.
- [] No comparé String con ==.
- [] Los atributos están encapsulados.
- [] No concatené entradas en SQL.
- [] Cerré Connection, Statement y ResultSet.
- [] La transacción tiene commit y rollback.
- [] Validé en servidor.
- [] No expuse credenciales ni trazas.
- [] Separé web, servicio y persistencia.
- [] Documenté cómo ejecutar.
- [] Puedo explicar una decisión y un riesgo.

Cómo presentar lo incompleto

"Implementé el flujo principal y dejé compilando la solución. Por tiempo no terminé el control CSRF. Antes de producción lo agregaría como filtro con token de sesión y una prueba de POST sin token. También validaría el despliegue en la versión exacta de Tomcat."

Reconocer una brecha con un plan técnico preciso demuestra más criterio que ocultarla con código improvisado.